

Learning With AI

A NumPy-Based Multi-Layer Perceptron AI

Two Core Focuses:

- Data Manipulation for AI
- AI Accuracy

Concept 1

Temporal Analysis & Signal Processing

The System as a Signal

Most people treat server logs as plain text. We treat them as something more powerful:

Stochastic Process : a sequence of random variables evolving over time.

What Is It? The Stochastic Process

A **Stochastic Process** models a system as a stream of values that change randomly over time — not randomly in the sense of "meaningless," but in the sense of "probabilistic and measurable."

- Each metric (CPU, memory, error rate) is a random variable X_t
- Together they form a sequence: $X_1, X_2, X_3 \dots X_t$
- We analyze how those values evolve not just what they are

What I Learned: Weight Drift

"Normal" is a moving target.

A CPU spike at **3 AM** is an anomaly. The same spike at **3 PM** (peak hours) is expected behavior.

Static thresholds like "*alert if CPU > 80%*" are mathematically insufficient for dynamic systems.

The model must continuously redefine its baseline as the system changes.

How We Implemented: Sliding Traces

To give the model memory, we used a **Sliding Trace** approach.

Instead of one value at a time, the model receives a chunk of consecutive values — letting it see **10 seconds of history at once**.

```
traces = numpy.lib.stride_tricks.sliding_window_view(stream, trace_size)
```

A 1D telemetry stream → a 2D matrix of time traces → fed into the MLP.

Concept 2

Standardizing the Signal

Before feeding data into a neural network, we need all metrics on the **same scale**.
A CPU reading of 80 and a memory reading of 80 may mean very different things.

What Is It? The Z-Score

The Z-Score answers: *"How far is this value from normal, in standard deviations?"*

$$z = \frac{x - \mu}{\sigma}$$

- $z = 0 \rightarrow$ perfectly average
- $z = 3 \rightarrow$ three standard deviations above normal, likely an anomaly
- Works across any metric, any unit, any scale

What I Learned: Preventing Gradient Explosion

Without normalization, large raw values cause the model's weight updates to spiral out of control during training — known as **Gradient Explosion**.

Z-Score normalization centers the data at 0, keeping gradients stable and training reliable.

How We Implemented: Real-Time Z-Score in NumPy

```
z = (x - np.mean(trace)) / np.std(trace)
```

Calculated on every incoming trace in real-time —
no stored global mean, no batch processing required.

Concept 3

Frequency Domain Analysis

Some patterns are completely invisible when you look at raw values over time. They only appear when you ask a different question:

"What frequencies are hidden inside this signal?"

What Is It? Fourier Transforms

A **Fourier Transform** decomposes a signal into its constituent frequencies — like splitting white light through a prism to reveal the colors inside.

Applied to system telemetry, it can expose:

- **Periodic background tasks** (a heartbeat of regular activity)
- **Cyclic load patterns** (daily or hourly rhythms)
- **Hidden anomalies** buried under surface-level noise

What I Learned: Separating Noise from Signal

Not all unusual activity is an incident. Systems produce constant low-level noise — scheduled jobs, health checks, background scans.

Frequency analysis taught me to distinguish:

- **System Noise** — high-frequency, low-impact background chatter
- **Signal** — low-frequency, high-impact state changes that indicate real incidents

How We Implemented: NumPy FFT

```
frequencies = np.fft.fft(trace)
```

By analyzing log-frequency spikes in the frequency domain, the model learned to **ignore high-frequency chatter** and focus only on the low-frequency shifts that signal a real incident.

Summary: Temporal Feature Pipeline

Stage	What Happens
Raw Data	Unprocessed telemetry stream
Tracing	Sliding trace via <code>stride_tricks</code>
Scaling	Z-score normalization
FFT	Frequency domain transformation
Feature Vector	MLP-ready input

Pipeline: Raw Data → Tracing → Scaling → FFT → Feature Vector

Concept 4

Probability & Information Theory

How do you measure how *weird* a system state is?

Information Theory gives us the math to do exactly that — turning the vague feeling of "something seems off" into a precise, calculable number.

What Is It? Shannon Entropy

Shannon Entropy measures the average uncertainty — or "chaos" — in a stream of data.

$$H(X) = - \sum P(x) \log P(x)$$

- Low entropy → system is predictable, behaving normally
- High entropy → system is chaotic, logs are unpredictable
- A sudden **spike in entropy** is a mathematical signature of an incident

What I Learned: An Incident Is a Spike in Chaos

When a system crashes, its logs stop following familiar patterns. Error types multiply, sequences break down, and the data becomes highly unpredictable.

That unpredictability *is* the incident — and entropy measures it directly. We're not looking for specific error messages; we're measuring **disorder itself**.

How We Implemented: Entropy Scoring

For each incoming trace, we calculate the probability distribution of log types. If the entropy of that distribution exceeds a dynamic threshold:

$H(\text{trace}) > \text{threshold} \rightarrow \text{MLP flags a high-priority incident}$

No hardcoded rules. No keyword matching. Pure math.

Concept 5

Probabilistic Classification

A binary alert system — *incident* or *no incident* — throws away valuable information. We need to know not just *if* something is wrong, but *what* is wrong and *how confident* we are.

The solution: replace binary outputs with a **probability distribution**.

What Is It? The Softmax Function

Softmax converts the raw outputs of the neural network (called logits) into a probability distribution across all possible incident types.

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$$

- Every output is between 0 and 1
- All outputs sum to exactly 1.0
- The model expresses *confidence*, not just a yes/no decision

What I Learned: Using Confidence Scores

The probability score itself carries meaning:

- Model returns **0.51** → flag as "Uncertain" — escalate for human review
- Model returns **0.99** → flag as "Critical Outage" — trigger automated response

This nuance dramatically reduces false positives compared to binary classification.

How We Implemented: Custom Output Layer

```
exp_scores = np.exp(logits)
softmax = exp_scores / np.sum(exp_scores)
```

Built as a custom `OutputLayer` class in pure NumPy —
producing a probability distribution across **12 incident types**.

Concept 6

Learning From Mistakes

A model that never updates is useless. The question is: *how does a neural network know when it's wrong, and how does it correct itself?*

The answer lies in measuring the gap between what the model predicted and what actually happened — and using that gap to drive learning.

What Is It? Cross-Entropy Loss

Cross-Entropy Loss is the cost function that penalizes the model for being *confidently wrong*.

$$L = - \sum y \cdot \log(\hat{y})$$

- If the model predicts the right class with high confidence → loss is near 0
- If the model predicts the wrong class with high confidence → loss is very large
- The goal of training is to minimize this loss

How We Implemented: Backpropagation

```
delta = predicted_probs  
delta[correct_class] -= 1
```

We calculate the derivative of the loss and propagate it **backward** through every layer

—

adjusting each weight to "punish" the neurons responsible for the wrong prediction.

This is the core of how the network learns.

Results

By combining **Stochastic Time-Series Analysis** with **Information Theory**:

98.42% Accuracy Rate

The math of "surprise" — entropy, probability, and loss —
proved to be the most effective way to catch system failures automatically.